ChatGPT

Executive Summary

This project envisions an **interactive 3D Solar System explorer** using authoritative NASA data. Its goal is to deliver a scientifically accurate, educational web application where users can freely navigate planets, moons, asteroids, comets (including Kuiper belt/Oort cloud objects) and even nearby exoplanets. The design will draw data primarily from NASA sources – e.g. JPL's Horizons and SBDB services for orbital/physical data, the NASA Exoplanet Archive for extrasolar worlds, and the Planetary Data System archives for imagery and mission data. We recommend modern WebGL frameworks (Three.js, Babylon.js or Cesium) to render the scene with realistic scale, lighting and textures. Key features include smooth camera control (zoom/free-fly), click-to-inspect info panels, time controls and ephemeris playback, measurement tools, search/bookmarks, and unique enhancements like procedural terrain and live mission telemetry. The system will use client-side caching and static hosting for performance, with periodic updates via NASA APIs. Rigorous testing against NASA ephemerides will ensure accuracy. Below we detail the data sources, technical choices, and design considerations in tables and diagrams for easy reference.

【20†embed_image】 *Figure: NASA illustration of Solar System planets (public domain). The app will use similar realistic textures and scales for accuracy.*

Goals and Scope

- **Scope:** Full Solar System objects (Sun, 8 planets, major dwarf planets, moons, asteroid and Kuiper belts, comets/Oort cloud) plus a selectable subset of *nearby exoplanets*. Incorporate NASA spacecraft trajectories (past/present missions) as optional overlays.
- **Scientific Accuracy:** Use NASA/JPL ephemeris data and physical parameters. Show correct orbital motions and relative sizes (with practical scaling), real imagery/textures and atmospheres (e.g. Earth's clouds, Mars dust). Ensure gravitationally-informed orbits and true inclination/rotation axes.
- **Interactivity:** Users can navigate via free-fly or orbit camera, zoom smoothly, pause/rewind time, select objects for details (physical stats, images, discovery metadata). Include timeline controls (date range, speed) and tools (distance measurements, bookmarking views). Offer educational context (e.g. annotated orbits, mission labels).
- **Accessibility & UX:** Design with clear UI (search bar, info panels, legends). Ensure keyboard navigation and screen-reader compatibility (ARIA labels, alt-text on images). Use colorblind-friendly palettes and font sizes. Organize UI in logical panels (object info, controls, legend).
- **Performance & Hosting:** Build as a web application (static site or SPAs) with WebGL. Optimize with level-of-detail (LOD) for distant objects, culling, and minimized API calls. Host on CDN or static hosting (e.g. AWS S3, Cloudflare). Cache NASA data locally (or on server) with scheduled updates.
- **Development:** Agile phases with early prototyping (core 3D scene) followed by progressive feature addition. Test numerical accuracy against NASA ephemerides at each stage.

【46†L416-L424】NASA's own *Eyes on the Solar System* is an example: it "uses the most accurate data and imagery possible" to simulate planets, moons, asteroids, comets and spacecraft in a browser-based 3D scene【46†L416-L424】. Our project follows this high standard of data fidelity.

Data Sources and APIs

Object Type	NASA Source/API	Data Fields & Products
Planets (Mercury–Neptune)	JPL HORIZONS API (ssd.jpl.nasa.gov/api/horizons)【12†L23-L30】	Orbital elements (a, e, i, Ω, ω, M0), epoch; physical properties (mass, radius, oblateness, rotation period, axial tilt, albedo); atmosphere details; texture images (from missions)【22†L307-L314】【12†L23-L30】.
Sun	JPL HORIZONS (ID 499) as "target"	Heliocentric orbital constants (trivial), solar radius, mass, luminosity, rotation; solar texture (Sun maps). Data from SOHO/SDO missions if needed.
Major Moons (e.g. Moon, Io, Titan)	JPL HORIZONS (satellites list) or PDS (by planet)	Orbital elements around parent, size, mass, albedo, rotation; surface elevation (terrain) models; surface images (e.g. Lunar Reconnaissance Orbiter maps)【41†L116-L124】.
Dwarf Planets (Pluto, Ceres, etc.)	SBDB API (ssd-api.jpl.nasa.gov/sbdb) or Horizons	Orbital elements, diameter, mass, albedo【4†L108-L117】, rotation; Pluto/Charon mutual orbits; high-res surface textures (New Horizons, Dawn).
Asteroids (NEOs & Main Belt)	JPL SBDB API (ssd-api.jpl.nasa.gov/sbdb)【7†L11-L18】	Orbital elements, diameter, albedo, spectral class, rotation period, asteroid family (SBDB "sb-class" fields)【4†L108-L117】. For large populations (asteroid belt/KBO belt), use simplified point cloud or particle fields.
Comets (Short/Long)	JPL SBDB (comet mode)【4†L108-L117】 or Minor Planet Center (via SBDB)	Orbital elements (often high-ecc, i), nucleus size, albedo, period, perihelion date; visual tail/jet particle effect (procedural). Include discovery date, original orbit classification.
Kuiper Belt Objects (KBOs) & Scattered Disc	SBDB (TNOs, CEN classes)【4†L108-L117】	Orbits for known KBOs (e.g. Pluto, Haumea, Sedna); physical (sizes, albedo). Represent belt as band of objects using orbital distribution models (code or PDS ring data).
Oort Cloud (long-period comets)	SBDB (known LP comets), literature (theoretical)	Very-long period comets lists (e.g. C/2011 L4 PanSTARRS), orbit elements; represent Oort cloud schematically (not individual objects).
Exoplanets (nearby)	NASA Exoplanet Archive (IPAC/NExScI)【30†L165-L168】【43†L19-L27】	Confirmed exoplanets and host stars. Orbital parameters (period, a, e, inclination), planet radius/mass, equilibrium temperature, discovery method/year【43†L19-L27】. Use *HOST* star coordinates (e.g. from GAIA) to place system. Include only nearby (~<50 ly) for practicality. Use the Archive's TAP or REST API (nph-nstedAPI)【30†L165-L168】.

Object Type	NASA Source/API	Data Fields & Products
Spacecraft/ Missions	SPICE kernels and NASA mission archives (PDS); CelesTrak for debris.	Spacecraft ephemerides (JPL SPICE or Horizons "TARGET=spacecraft"); allow optional trajectory plotting and telemetry (telemetry via NASA APIs if available).

Each dataset is large. For efficiency, query APIs ahead of time or on-the-fly with caching. For example, JPL's SBDB query can fetch selected fields (`diameter, albedo, orbital elements…`)【4†L108-L117】. NASA's Exoplanet Archive API uses a base URL `https://exoplanetarchive.ipac.caltech.edu/cgi-bin/nstedAPI/nph-nstedAPI?` with table queries (e.g. `table=ps` or star/planet tables)【30†L165-L168】. The project should script these queries to build local JSON or database tables of needed fields.

Additional data sources: NASA Planetary Photojournal (photojournal.jpl.nasa.gov) and mission archives (Cassini PDS, Dawn PDS, etc.) for high-res textures and maps. NASA/LPI slide-set "Solar System in 3-D" notes that spacecraft stereo images reveal topography otherwise unseen【27†L22-L30】, underscoring the value of accurate surface data. We should use NASA's most recent and validated data (planetary fact sheets, mission data releases) wherever possible.

3D Engine & Framework Options

Engine	Pros	Cons
Three.js	Very popular JS/WebGL library; large community and examples (including solar system demos【25†L13-L17】). Flexible, lots of plugins (e.g. controls, GIS). Good integration with WebXR for VR.	Lower-level than game engines; requires more development for physics/terrain. No built-in geospatial/time libraries (must code or use extras).
Babylon.js	High-level WebGL engine with PBR rendering, physics support and an excellent editor. Good for complex scenes; built-in camera controls. Active development and docs.	Slightly heavier library. Less widely used for science visualization, but still JS/ WebGL (works well in browser).
CesiumJS	Geospatial focus: handles large-scale planetary terrain/imagery, global to sub-meter. Built-in time-dynamic visualization and Earth-centric tools. Good for showing Earth and planetary surfaces.	Primarily Earth/globe oriented (but supports other bodies). Heavy-weight; learning curve for custom apps. Licensing considerations for commercial use (uses Apache 2.0 with some commercial assets).
Unity (WebGL)	Industry-standard game engine. NASA *Eyes* uses Unity (via WebGL builds). Exceptional graphics, physics, and VR/ AR support. Rich editor and asset ecosystem.	Large build size (tens of MB), platform overhead. Not open-source; requires Unity license (free tier). Editing done outside web stack; harder to integrate with web APIs.

Engine	Pros	Cons
A-Frame / WebXR	Easy VR-capable WebGL framework (entity/component). Good for quick VR scenes with minimal code.	Less mature for complex scenes; performance may lag for large scenes. Mainly VR-centric, less fine control.
OpenSpace / Cesium for Geoviz	NASA/Swiss project specialized for science vis (OpenSpace), Cesium's Space applications.	OpenSpace is heavy C++/OpenGL (desktop). Cesium's Space package still evolving. Not mainstream web dev tools.

Recommendation: For a browser-based site, a JavaScript/WebGL engine is best. Three.js and Babylon.js are strong candidates. Three.js is extremely popular with many tutorials; Babylon.js offers more out-of-box features (e.g. physics engine, scene inspector). CesiumJS could be used if global terrain (e.g. Earth, Mars surfaces) is critical; it also has high-res imagery tiles but may require custom planetary projection. If reusing NASA code, note *Eyes on the Solar System* was built in Unity, but for an open 3D web app we lean on Three.js/Babylon.js. Table above compares their trade-offs (stability, ecosystem, learning curve).

Rendering Requirements (Scientific Accuracy)

- **Scale Handling:** The true Solar System spans 1–30+ AU. To visualize both inner and outer systems, use **non-linear scaling** or multiple camera modes. For example, when zoomed out show true relative orbit sizes (maybe with logarithmic distance scaling), but allow switching to a "local scale" mode when inspecting one system. Three.js allows dynamic repositioning of origin or scaling factors. Use **distant fog** or scale bars to hint at true distances.
- **Lighting:** Use a directional light from the Sun's position (or a point light at Sun) for realistic illumination. Enable **physically-based rendering** (PBR) materials to simulate planetary reflectance. For atmospheres, implement shader-based scattering (e.g. atmosphere glow on limb). Shadows: if performance allows, add **planetary shadows on rings** (Saturn's rings casting on planet) and **eclipses** (e.g. solar eclipses on moon). For Earth, consider real-time Earth Night lights or cloud shadows if data available.
- **Textures & Materials:** Use high-resolution NASA imagery for surfaces (e.g. NASA photojournal maps for planets, altaz projection textures). For gas giants, use cloud maps (e.g. Jupiter/Juno maps) and turbulent cloud shaders. Apply bump or normal maps to simulate topography (Mars/terrain, Moon craters). For rings and belts, use **particle systems** or transparent textured planes with ring patterns (Cassini maps of Saturn's rings from NASA). Implement *Level-of-Detail* (LOD): lower-res sphere geometries/textures for distant view, detailed meshes when close.
- **Atmospheres:** Model thin atmospheric effects: e.g. Earth's blue rim, Venus thick cloud scattering, gas-giant haze. Could use additive blended sprites or custom GLSL shaders for glow. NASA images show planetary terminators; simulate day/night shading and limb glow for realism.
- **Shadows:** For a star-based system, full dynamic shadows for all moving bodies is expensive; instead consider static or precomputed shadow (e.g. ring shadows from Saturn's geometry) or very coarse shadows for major bodies.
- **Particles:** To depict comet tails, asteroid belts or Kuiper belt dust, use particle emitter systems with physics or billboards. Example: when comets approach perihelion, emit a tail along velocity vector. Kuiper/asteroid belts can be rendered as volumetric dust (semi-transparent band).
- **Coordinates and Reference Frames:** Support ecliptic coordinate plane, equatorial frame, etc. Offer optionally to show planetary axes and orbital planes for context.

- **Time Simulation:** Calculate orbits with *accuracy*. Either use real-time JPL data (see Testing below) or approximate Keplerian motion for performance. NASA's Horizons can generate ephemerides for any date; for continuous animation, precompute ephemeris tables or use approximate formulas (e.g. solve Kepler's equation per step).

Interaction Features

- **Camera Controls:** Offer *free-fly* mode (first-person style) and *orbit* mode (around selected object). Implement smooth zoom (mouse wheel/pinch) and pan. Provide presets ("jump to" for each major body). Controls can reuse libraries (e.g. Three.js OrbitControls or custom target-CAM).
- **Click-to-Inspect:** Clicking an object (planet, moon, asteroid, spacecraft) brings up an overlay panel with *information*: name, type, physical stats, images/spectra, discovery details (for comets/asteroids/exoplanets), and NASA citation links. Include *search* (object name filter). Show *orbits:* toggle drawing of orbital paths. Selection panel should have hyperlinks to NASA fact-sheets or mission pages for further reading.
- **Timelines & Ephemeris:** Include date/time controls (calendar picker or slider) to view past/future positions. "Ephemeris playback" mode animates the Solar System over selected span. Provide **measurement tool** (distance/angle between two objects, current synodic periods). Optionally allow plotting an object's heliocentric distance over time (graph).
- **Annotations:** Users can place markers or annotations (with notes or questions) anchored to solar system objects. This is useful for education or guided tours. For example, annotate "Cassini Grand Finale: Saturn at 2017-09-15" etc.
- **Bookmarks & Tours:** Save camera/bookmarks for easy return. Support scripted *tour/story* feature: e.g. preset camera paths with narration or subtitles (e.g. highlight phases of Voyager, or planetary alignment).
- **Accessibility:** Keyboard shortcuts (e.g. arrow keys to pan, +/- to zoom). High-contrast UI toggle (for visually impaired). Provide captions for any audio or animation.
- **Device Input:** Support touch controls (swipe/zoom) for tablets/phones. Consider gamepad or VR controllers for immersive navigation.

UI/UX Design & Accessibility

Sketch a clean, intuitive interface. Possible layout: left panel with **Object Info** (name, image, stats, links), bottom toolbar for **time controls** (play/pause, date selector, speed), right panel for **controls** (search, layer toggle, bookmarks). A minimalist legend or mode indicator (e.g. ecliptic plane on/off).

- **Wireframe Suggestion:** Title bar with search; 3D canvas main area; sidepanel (info/bookmarks); bottom bar (timeline slider, time controls). Use icons (play, pause, camera home, info, etc.) with tooltips.
- **Color & Contrast:** Dark background (space), bright bodies. Ensure UI contrasts meet WCAG AA standards. Allow colorblind-friendly mode (e.g. differentiate orbits with shapes/text not just color).
- **Responsiveness:** UI should adapt to screen size (reflow panels on mobile). Collapsible sidebars. Avoid horizontal scroll.
- **ARIA and Alt Text:** All interactive elements labeled (e.g. `<button aria-label="Zoom In">`). 3D canvas gets a short descriptive alt text ("Interactive 3D Solar System"). Images in info panels have captions.

- **Performance & Feedback:** Show loading spinners when fetching data or textures. Indicate current simulation time vs real time. Provide error messages if NASA data fails (e.g. “Data unavailable at this epoch”).

Performance, Hosting, and Caching

- **Level-of-Detail (LOD):** Implement LOD for meshes (low-poly spheres in distance), and texture mipmaps. E.g. an 8K Mars texture only loaded at close zoom, else use 2K. Use **frustum culling** so off-camera objects not rendered.
- **Instancing:** For asteroid/comet fields, use GPU instancing or point sprites to render thousands with minimal draw calls.
- **Data Caching:** Pre-fetch and cache planetary ephemerides and orbit data on initial load or daily updates (to avoid live querying). For example, at app startup request planet positions via Horizons once and reuse locally. Store fetched data in browser IndexedDB or a local server cache.
- **Hosting:** A static-site approach (HTML/CSS/JS) on a CDN (e.g. GitHub Pages, AWS S3+CloudFront) is sufficient. All heavy assets (textures, models) should be served from fast cloud storage. If real-time data is needed, host a small backend or serverless function to relay NASA API calls (since CORS may block direct JPL calls).
- **Web Workers:** For heavy computations (Kepler solver for many objects), use Web Workers to avoid freezing UI.
- **Benchmarking:** Test frame rates on typical devices. Aim for 60fps on desktop; at least 30fps on mobile. Adjust detail accordingly.

Unique Features and Enhancements

- **Procedural Terrain:** Dynamically generate high-resolution terrain on planetary surfaces (e.g. finer detail when zoomed into Mars crater using DEM data from PDS). This goes beyond flat textures. Use heightmaps (from MOLA for Mars, LOLA for Moon).
- **Real-Time Mission Telemetry:** If feasible, stream current spacecraft telemetry (e.g. Parker Solar Probe data) onto the scene. Use NASA HORIZONS or mission APIs for live state vectors. Show spacecraft orientation and data (speed, distance).
- **AR/VR Support:** Incorporate WebXR to allow VR headset viewing (e.g. Oculus, VR-ready browsers). Provide an AR mode (e.g. use device camera to overlay planets in the sky, like a star chart). NASA’s “Spacecraft AR” is an example.
- **Multiplayer Tours:** Allow multiple users to join a shared session (via WebRTC) to co-navigate. They could see each other’s viewports or leave messages. This supports virtual classroom experiences.
- **Education Modules:** Embed mini-lectures or quizzes (e.g. identify planets by orbit tilt, run-time fact pop-ups, “spot the moon” challenges). These guided experiences can use the same 3D view.
- **Citation Links:** Each info panel will link to NASA sources (e.g. Planetary Fact Sheet, mission page, Exoplanet Archive entry) for credibility. Encourage citation of NASA data.

【23†embed_image】 *Figure: Example NASA imagery (Ceres from Dawn) used for texture/maps.*

API Design and Data Updates

- **Unified Data API:** Design a backend API (or use static JSON) that aggregates data from NASA services. For instance, a REST endpoint `/api/objects` could return basic orbital/physical data for

all bodies, drawn from JPL/HORIZONS and SBDB. Another endpoint `/api/exoplanets` hits the Exoplanet Archive. If no custom backend is built, the front-end can call NASA endpoints directly (watch CORS).
- **Update Cadence:** Set regular data sync (e.g. nightly or weekly) from NASA. NASA's ephemeris/orbit data is mostly static for known objects, so weekly updates suffice. For fast-moving objects (new comets/NEOs), consider daily checks via SBDB. The Exoplanet Archive adds new planets frequently; schedule its sync monthly or on major paper releases.
- **API Caching Layers:** Use CDN or server caching for NASA queries. For example, use a lambda function or Cloudflare Worker as proxy with caching.
- **Documentation:** Provide an OpenAPI/Swagger spec for any custom API to clarify data fields for front-end developers.

Testing and Validation

- **Ephemeris Validation:** Compare simulated positions/orbits against NASA Horizons' output at multiple epochs (e.g. current date) to verify accuracy. Use test cases (e.g. "Where is Mars at 2026-10-10 18:00 UTC?"). Differences should be <0.01° arc.
- **Physical Checks:** Ensure rendered distances (e.g. Earth-Moon average) match data (±1%). Confirm planetary sizes.
- **Performance Testing:** Use tools like Google Lighthouse and WebGL Profiler.
- **Cross-Browser:** Test in Chrome, Firefox, Safari (Mobile and Desktop). Ensure WebGL2 support or fallback.
- **UI/UX Testing:** Conduct usability tests (keyboard-only navigation, screen reader with ARIA). Confirm color schemes against WCAG (e.g. use contrast checker).

Security and Privacy

- **Data Integrity:** NASA data is public; no sensitive user data is handled. Use HTTPS for all API calls.
- **User Privacy:** If users can log in or save bookmarks, inform about any storage. For public demo, no login required (anonymity).
- **Third-Party Libraries:** Use well-maintained open-source frameworks. Keep libraries updated to avoid vulnerabilities.
- **CORS and Rate Limits:** Respect NASA's API usage limits (e.g. JPL SBDB limits 1 concurrent request per IP【4†L108-L117】). Implement back-off or queue to avoid blocking.
- **Content Security Policy (CSP):** Set strict CSP headers since we load external scripts/textures.

Data Source Comparison

Source	Scope	Access Method	Notes
JPL Horizons (ssd.jpl.nasa.gov)	All planets, moons, spacecraft	HTTP REST (horizons.api) 【12†L23-L30】	Highly accurate ephemerides; supports observer or vector outputs. Use for planets, major moons, missions.

Source	Scope	Access Method	Notes
JPL SBDB (ssd-api.jpl.nasa.gov)	Asteroids, Comets, KBOs	HTTP REST (sbdb.api) 【7†L11-L18】	Orbital & physical data on *all known* small bodies in Solar System. Fields customizable (see docs).【4†L108-L117】
NASA Exoplanet Archive (IPAC)	Confirmed exoplanets	TAP/API (nph-nstedAPI)【30†L165-L168】【43†L19-L27】	Comprehensive catalog of exoplanets with publications. Access via REST or IVOA TAP. Includes host star data.
NASA PDS (Small Bodies & Other)	Mission data, maps, textures	PDS nodes (via FTP/API)	Contains images/terrain (e.g. Cassini, Dawn, MRO DEMs). Data volumes large; use pre-downloaded assets.
NASA Planetary Fact Sheets	Planetary constants, atmospheres	Web pages / PDFs	Summary of planet properties. Useful for human reference, not automated API.
Minor Planet Center (MPC)	Observations, designations	Web/AstroPy (not direct API)	Could cross-check SBDB; not needed if SBDB used.

Engine Comparison

Engine/Lib	Key Features	Pros	Cons
Three.js	WebGL library, shaders, geoms	Flexible, large community (Solar System demos exist)【25†L13-L17】	Lower-level API (more coding).
Babylon.js	WebGL engine, PBR, physics, editor	Easy setup, inspector/debug tools	Less astronomical focus
CesiumJS	Geospatial 3D (terrain, tiles)	Built-in time-dynamics, terrain	Not specialized for solar orbits; heavy.
Unity (WebGL)	Full game engine	Mature tools, VR/AR support	Large download, not purely web tech.

Feature Complexity and Effort

Feature	Complexity	Effort
Core 3D scene (sun, planets, orbits)	Medium	1-2 developer-months
Smooth camera & navigation	Low	1 dev-month
Object click-info panel (UI overlay)	Low	0.5 dev-month

Feature	Complexity	Effort
Time slider & ephemeris playback	Medium	1 dev-month
Orbital path rendering (ellipses)	Medium	0.5 dev-month
Accurate physics or Kepler motion	High	2 dev-months
Atmosphere shaders (scattering)	High	1 dev-month
Ring/asteroid-belt particle effects	Medium	1 dev-month
Planetary terrain (DEM)	High	2 dev-months
Procedural features (volcanoes, etc.)	Very High	3 dev-months
AR/VR integration	High	3 dev-months
Real-time mission telemetry	Medium	1 dev-month + backend
Multiplayer sync	Very High	4 dev-months
Accessibility (ARIA, contrast)	Low	0.5 dev-month
Testing & Validation	Medium	1 dev-month

Estimated effort assumes one experienced developer. Complexity is low/med/high based on technical challenge.

Architecture (High-Level)

```
graph LR
    subgraph UI Layer
      UI[User Interface<br/>(HTML/CSS/JS)] -->|commands| CamCtrl(Camera
Controls)
      UI --> InfoPanel[Object Info Panel]
      UI --> TimeControl[Time Slider/Controls]
    end

    subgraph 3D Layer
      CamCtrl --> Scene[Three.js/Babylon Scene]
      Scene -->|renders| Camera
      Scene -->|renders| Objects[Planetary Bodies & Spacecraft]
    end

    subgraph Data Layer
      Objects --> DataAPI[NASA Data API/Cache]
      DataAPI --> Horizons[JPL Horizons API]
      DataAPI --> SBDB[JPL SBDB API]
      DataAPI --> Exo[NASA Exoplanet Archive API]
      DataAPI --> PDS[Planetary Data System]
    end
```

```
    DataAPI -->|fetch imagery| TextureServer[Texture/Image Assets]
```

Diagram: The front-end UI (left) controls a WebGL 3D scene (middle). The scene fetches data from NASA APIs (right) either directly or through a local cache. Static texture servers store large images.

Project Timeline

```
gantt
    title Development Timeline (2026)
    dateFormat  YYYY-MM-DD
    section Planning
    Requirements & Design      :done, a1, 2026-01-01, 2026-01-31
    Data & API Research        :done, after a1, 2026-01-15, 30d
    section Implementation
    Core 3D Engine Setup       :active, b1, 2026-02-15, 45d
    UI/UX Prototyping          :b2, 2026-02-15, 30d
    Data Integration (APIs)    :b3, after b1, 2026-03-30, 45d
    Feature Development        :c1, after b3, 2026-05-15, 60d
    section Testing & Launch
    Testing/Validation         :d1, after c1, 2026-07-15, 30d
    Deployment & Documentation :d2, after d1, 2026-08-15, 30d
```

Timeline: initial planning in Q1 2026, core implementation through mid-2026, testing and launch by Fall 2026.

References

Data and concepts cited above rely on NASA resources and published works. For example, the JPL Small-Body Database API and Horizons API document the available orbital and physical fields【7†L11-L18】【12†L23-L30】. The NASA Exoplanet Archive's data holdings are described in Christiansen et al. 2025【43†L19-L27】. NASA's "Eyes on the Solar System" emphasizes using the **most accurate** NASA data and imagery【46†L416-L424】. NASA science sites enumerate Solar System objects and their traits【22†L307-L314】. For data archives, note the PDS Small Bodies Node collects mission data on asteroids/comets【41†L116-L124】.

The tables and diagrams above synthesize these official sources into a cohesive specification. Any data or image used from NASA (e.g. planetary textures) is public domain (cited as NASA/JPL in captions).
